

# Distributed Systems Handin 3

---

BSDISYS1KU Carl Philip Steuch [caps@itu.dk](mailto:caps@itu.dk) Johannes Jensen [johje@itu.dk](mailto:johje@itu.dk) 29-10-2025

<https://github.com/Joha6210/DistributedSystems-Handin3>

## Instructions on how to run the server and clients

To run the server use the following command:

```
go run server/server.go [:PORT]
```

ex. To open a server listening on port :5050

```
go run server/server.go :5050
```

To create and connect with a client use the following command:

```
go run client/client.go [USERNAME] [:PORT]
```

ex. To create a client with the username Alice and create a connection to the server on port :5050

```
go run client/client.go Alice :5050
```

# 1. Streaming Model Selection

- In our chit chat implementation we use server-side streaming (1), to send multiple messages from the server to the client. This is done to support long-lived logical flow of data from the server to the clients (2)
- 

## 2. System Architecture

### 2.1 Overview

- **Architecture Type:**  
Chit chat is implemented as a server-client architecture, this allows for the server to serve multiple clients that can connect concurrently. Clients can leave and join whenever they like, and they will get the chat history send to them upon subscribing(connecting) to the server.

### 2.2 Components

- **Server:** The server is responsible for letting clients 'subscribe', publish messages to chit chat and unsubscribe when the client wants to leave. The server also keeps track of the message history. The server will announce when a new client either joins or leaves the server.
- **Client:** Clients can connect to the chit chat server using the 'subscribe' method, this is for the client to "register" at the server, and allows for the client to receive the message history and new messages that are being published by other clients. Registered clients can also publish new messages to the chit chat server, for other clients to receive.

### 3. RPC Methods and Message Types

#### 3.1 Implemented RPC Methods

| RPC Method               | Type (Unary/Server Streaming/Client Streaming/Bidirectional) | Description                        |
|--------------------------|--------------------------------------------------------------|------------------------------------|
| PublishMessage (Message) | Unary                                                        | Publishes Message object to server |
| Subscribe (Client)       | Server Streaming                                             | Adds client to server              |
| Unsubscribe (Client)     | Unary                                                        | Removes client from server         |

#### 3.2 Message Types

| Message Name     | Purpose                                         | Fields                                                                                                        |
|------------------|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| message Message  | Contains the message and relevant information   | <code>string uuid = 1; string message = 2; int32 clock = 3; string username = 4; string timestamp = 5;</code> |
| message Response | Contains a response from the server to a client | <code>bool result = 1; int32 clock = 3;</code>                                                                |
| message Client   | Contains information about a client             | <code>string uuid = 1; string username = 2; int32 clock = 3;</code>                                           |

## 4. Lamport Timestamp Implementation

Anytime a client or a server receives a message, or is about to initialize an internal process, its internal Lamport Timestamp(LT) is incremented by 1.

To keep parity across participants, any communication between parties include the originators current LT, and the receiver adopts the larger between the received value and its own

on initialization do

```
t := 0 // each node has its own local variable t
end on
```

on request to send message m do

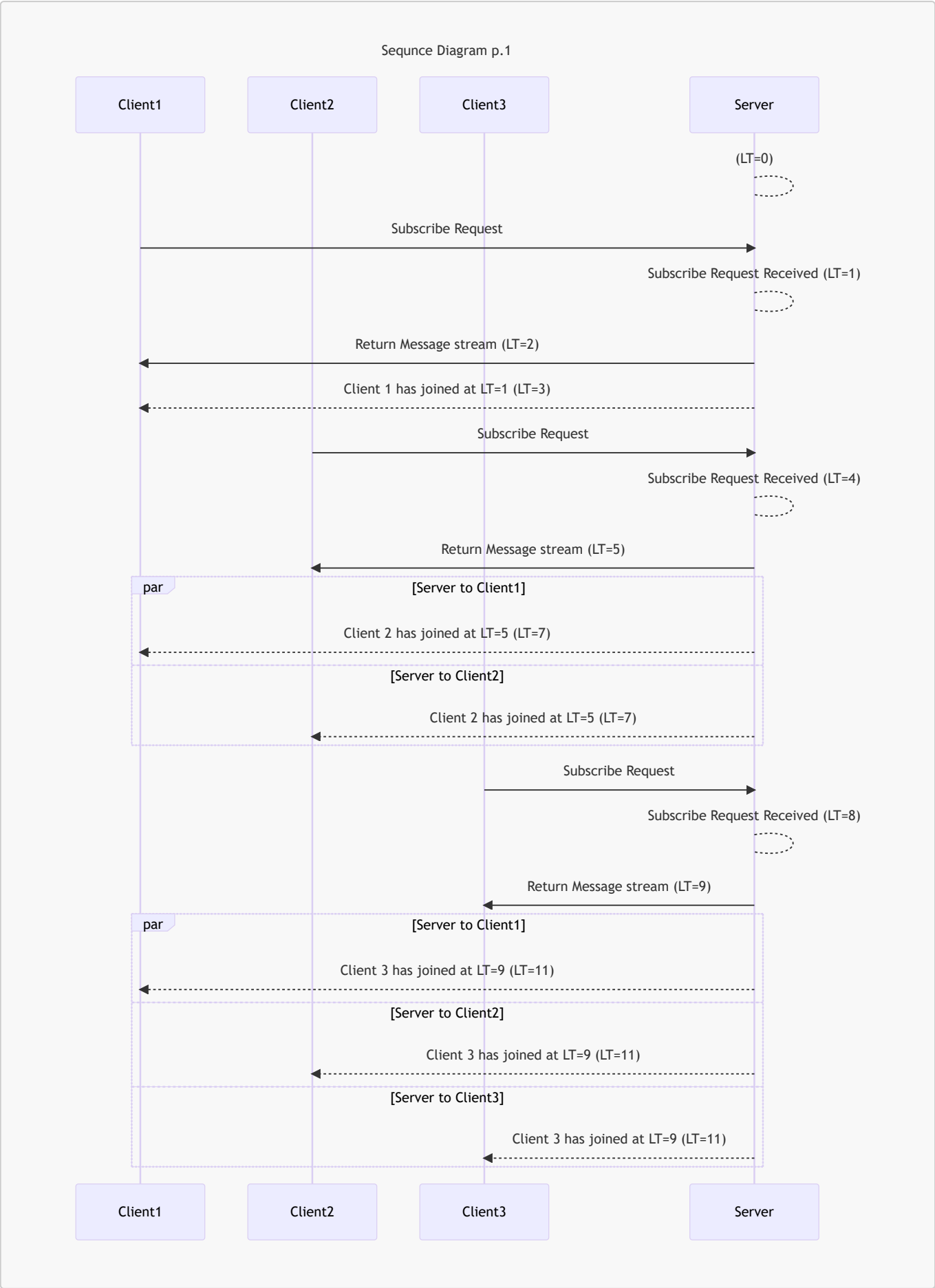
```
t := t + 1; send (t, m) via the underlying network link
end on
```

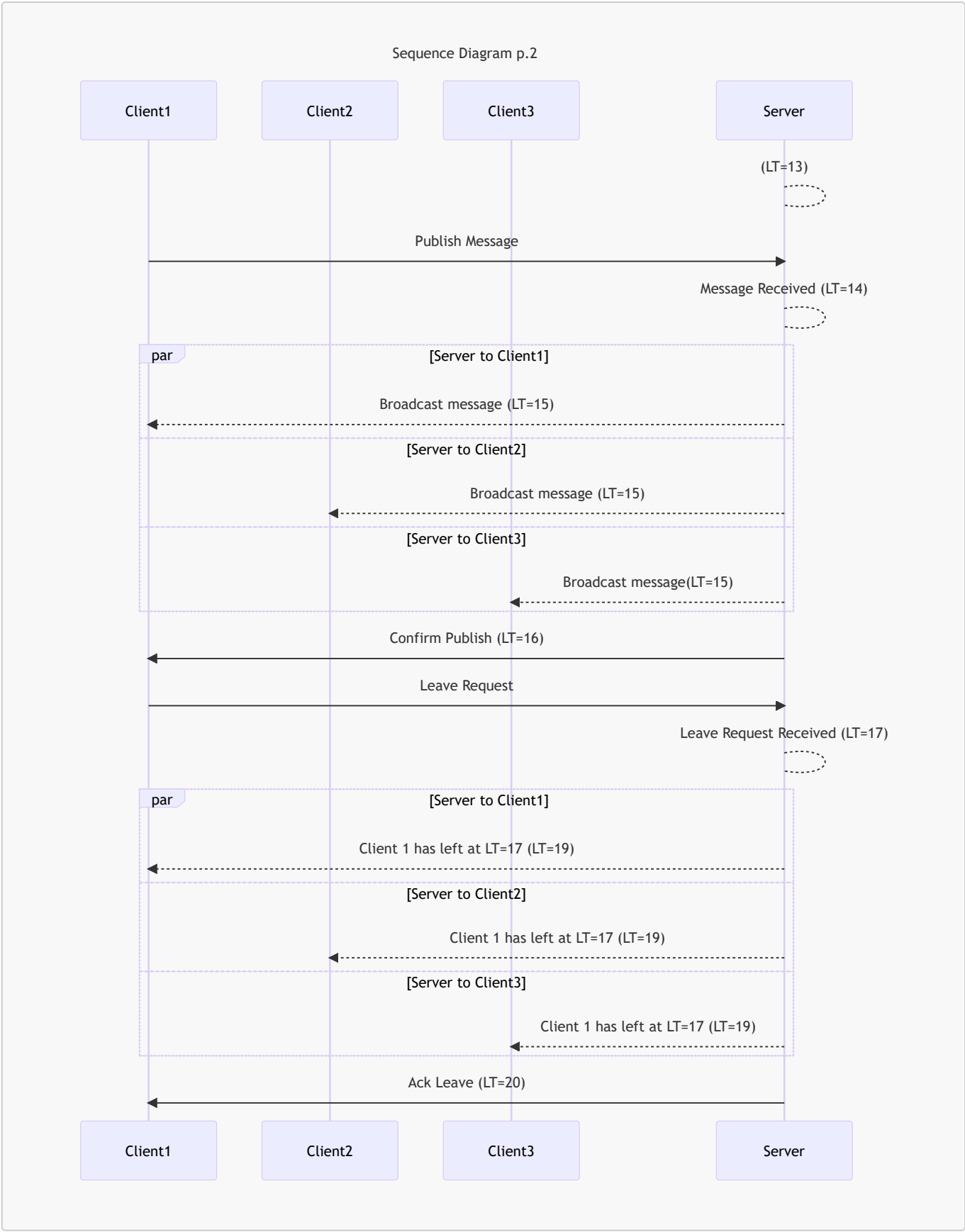
on receiving (t',m) via the underlying network link do

```
t := max(t, t') + 1
  deliver m to the application
end on
```

# 5. Sequence Diagram

LT refers to local server time, and is assumed to initialize at LT=0 for all parties.





# Appendix

---

## Logs

### Server

```
2025/10/28 11:02:03 gRPC server now listening on :5050... at logical time: 0
2025/10/28 11:02:03 Participant Alice joined Chit Chat at logical time 1
2025/10/28 11:02:03 [ @ 3] Server: Participant Alice joined Chit Chat at logical
time 1
2025/10/28 11:03:04 Participant John joined Chit Chat at logical time 5
2025/10/28 11:03:04 [ @ 7] Server: Participant John joined Chit Chat at logical
time 5
2025/10/28 11:05:09 Participant Bob joined Chit Chat at logical time 9
2025/10/28 11:05:09 [ @ 11] Server: Participant Bob joined Chit Chat at logical
time 9
2025/10/28 11:14:48 [28-10-2025 11:14:48 @ 15] Alice: Hej
2025/10/28 11:20:38 Participant Alice left Chit Chat at logical time 17
2025/10/28 11:20:38 [ @ 19] Server: Participant Alice left Chit Chat at logical
time 17
2025/10/28 11:20:38 Stream closed for Alice (60f45023-390a-4349-a86c-728ce70337fe)
at logical time: 21
2025/10/28 11:24:43 [28-10-2025 11:24:43 @ 23] John:
2025/10/28 11:24:43 Client disconnected: John (5a5131c1-5686-4738-85ab-
bff8345e2ffa) at logical time: 25
2025/10/28 11:24:44 [28-10-2025 11:24:44 @ 27] Bob:
2025/10/28 11:24:44 Client disconnected: Bob (8a73753e-3380-4919-a413-
f0c43ac4b11f) at logical time: 29
```

### Client 1 (Alice)

```
2025/10/28 11:02:03 [LT: 1] Subscribed successfully. Listening for messages...
2025/10/28 11:02:03 [ @ 5] Server: Participant Alice joined Chit Chat at logical
time 1
2025/10/28 11:03:04 [ @ 9] Server: Participant John joined Chit Chat at logical
time 5
2025/10/28 11:05:09 [ @ 13] Server: Participant Bob joined Chit Chat at logical
time 9
2025/10/28 11:14:48 [28-10-2025 11:14:48 @ 17] Alice: Hej
2025/10/28 11:20:38 [ @ 21] Server: Participant Alice left Chit Chat at logical
time 17
2025/10/28 11:20:38 [LT: 21] Server closed stream.
```

### Client 2 (John)



```
2025/10/28 11:03:04 [LT: 1] Subscribed successfully. Listening for messages...
2025/10/28 11:03:04 [ @ 5] Server: Participant Alice joined Chit Chat at logical
time 1
2025/10/28 11:03:04 [ @ 9] Server: Participant John joined Chit Chat at logical
time 5
2025/10/28 11:05:09 [ @ 13] Server: Participant Bob joined Chit Chat at logical
time 9
2025/10/28 11:14:48 [28-10-2025 11:14:48 @ 17] Alice: Hej
2025/10/28 11:20:38 [ @ 21] Server: Participant Alice left Chit Chat at logical
time 17
```

### Client 3 (Bob)

```
2025/10/28 11:05:09 [LT: 1] Subscribed successfully. Listening for messages...
2025/10/28 11:05:09 [ @ 5] Server: Participant Alice joined Chit Chat at logical
time 1
2025/10/28 11:05:09 [ @ 9] Server: Participant John joined Chit Chat at logical
time 5
2025/10/28 11:05:09 [ @ 13] Server: Participant Bob joined Chit Chat at logical
time 9
2025/10/28 11:14:48 [28-10-2025 11:14:48 @ 17] Alice: Hej
2025/10/28 11:20:38 [ @ 21] Server: Participant Alice left Chit Chat at logical
time 17
2025/10/28 11:24:43 [28-10-2025 11:24:43 @ 25] John:
```